HARAMAYA UNIVERSITY

College of Computing and Informatics • Department of Information Technology

Team: Asladin Abdikadir Abdii (0172/15) • Burka Jemal Mume (2807/16) • Tumsa Rameta (2613/16)

Course: Mobile App Development
Class: 4th Year, 1st Sem (Sec A)
Doc Type: Data & Retrieval Spec

All-Inclusive Database, Entity-Relationship & Retrieval Architecture

Scalable, Dynamic, Object-Oriented Data Model and Multi-Faceted Storage, Search, Sorting & Navigation Specification for **EthioIT ExitPrep**

1. Scalable, Dynamic & Object-Oriented Storage Architecture

To prepare graduating BSc Information Technology students for the national exit exam defined by the FDRE Ministry of Education National Exit Exam Blueprint for Information Technology (5 core themes, 13 selected courses, and 45 credit hours per the BSc IT Exit Exam Blueprint), **EthioIT ExitPrep** requires a database architecture that is **engine-agnostic, object-oriented, dynamically extensible, and offline-first** (Offline-First Sync Patterns). Rather than locking the application to a single database vendor, the data layer is designed around four core architectural pillars:

1. Object-Oriented Domain Encapsulation

Every curriculum unit inherits from a base `LearningEntity` class with shared identity, versioning (`version` , `updated_at`), priority, and difficulty attributes. Polymorphic content blocks (text, code, formulas, diagrams, flashcards) and encapsulated assessment options map cleanly to both object/document stores and relational tables.

2. Clear Entity-Relationship Semantics

Follows formal IBM Entity-Relationship Data Modeling and ERD Cardinality Standards to enforce referential integrity across Themes, Courses, Chapters, Study Materials, Questions, Options, and associative Content Relationships.

3. Dynamic Schema Extensibility

Supports new Ministry of Education blueprint revisions, new exam years, and custom pedagogical tags without breaking client builds through versioned entity contracts and an extensible `metadata_attributes{}` key-value map on every core entity.

4. Hybrid Cloud + Local Cross-Platform Storage

Pairs a cloud master database (Document or Relational, such as MongoDB Atlas or PostgreSQL) with a high-speed on-device relational/object store (SQLite with FTS5 via Drift or PowerSync) for instant offline access.

2. Complete Entity, Attribute & Relationship Specification (ER Model)

The logical data model separates **Curriculum Hierarchy Entities** (Themes, Courses, Chapters), **Learning & Assessment Entities** (Study Materials, Questions, Options), **Knowledge-Graph Relationship Entities** (Content Relationships), and **Student State Entities** (Progress, Bookmarks, and Exam Attempts).

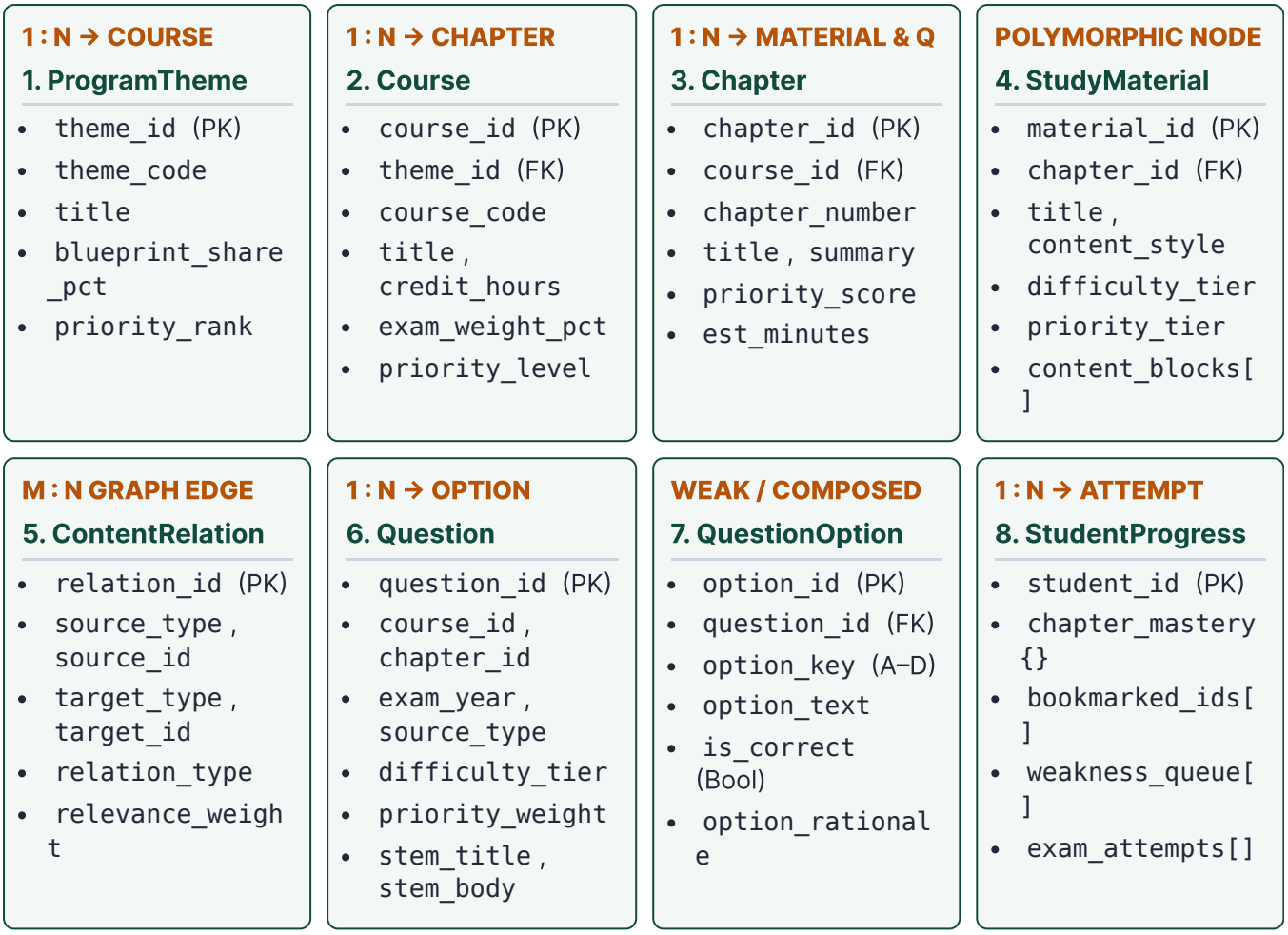


Figure 1: Logical Entity-Relationship (ER) diagram showing primary keys, foreign keys, and cardinalities across all 8 core entities.

Entity Name	Cardinality & Keys	Core Attributes & Domain Values	Role in Retrieval & Learning
ProgramTheme	`theme_id` (PK) `1 : N` with `Course`	`theme_code`, `title`, `blueprint_share_pct`, `priority_rank` (1–5)	Groups the 13 courses into the 5 national exit exam thematic areas.
Course	`course_id` (PK), `theme_id` (FK) `1 : N` with `Chapter`	`course_code`, `title`, `credit_hours`, `semester`, `exam_weight_pct`, `priority_level`	Filters study notes and exams by individual BSc IT course and blueprint weight.
Chapter	`chapter_id` (PK), `course_id` (FK) `1 : N` with `StudyMaterial`, `Question`	`chapter_number`, `title`, `sequence_order`, `objectives[]`, `priority_score` (1–100)	Organizes course syllabus into sequential chapters with priority rankings.
StudyMaterial	`material_id` (PK), `chapter_id` (FK) `M : N` via `ContentRelation`	`title`, `content_style` (`Note`, `Summary`, `Code`, `Flashcard`, `Formula`), `difficulty_tier` (`1=Simple`, `2=Moderate`, `3=Hard`), `priority_tier`	Delivers multi-style learning materials filtered by simplicity, hardness, or priority.

Entity Name	Cardinality & Keys	Core Attributes & Domain Values	Role in Retrieval & Learning
ContentRelation	`relation_id` (PK) `source_id` → `target_id`	`relation_type` (`PREREQUISITE_OF`, `RELATED_CONCEPT`, `TESTED_BY_QUESTION`, `EXPLAINS_ANSWER_FOR`), `weight`	Connects questions to exact study paragraphs and links prerequisite topics.
Question	`question_id` (PK), `course_id` (FK), `chapter_id` (FK) `1 : N` with `QuestionOption`	`stem_title`, `stem_body`, `code_block`, `exam_year` (`2015 – 2018 E.C.`, `Model`), `difficulty_tier` (`Simple`, `Moderate`, `Hard`), `priority_weight`, `bloom_domain`	Supports filtering by course, chapter, exam year, simplicity/hardness, and priority.
QuestionOption	`option_id` (PK), `question_id` (FK)	`option_key` (`A`, `B`, `C`, `D`), `option_text`, `is_correct`, `option_rationale`	Encapsulates choice text and explains why each specific option is right or wrong.
StudentProgress	`student_id` (PK) `1 : N` with `ExamAttempt`	`mastery_by_chapter{}`, `bookmarked_ids[]`, `confidence_index`, `weakness_priority_queue[]`	Powers adaptive navigation and personal priority sorting based on weak areas.

3. Object-Oriented Domain Model & Polymorphic Schema Specification

In the application layer, entities are modeled using object-oriented inheritance, composition, and polymorphism so the same Dart/TypeScript domain classes serialize seamlessly to both local relational tables (SQLite) and cloud document collections:

```
// Object-Oriented Domain Model (Polymorphic Learning Entities & Encapsulated
Relationships)
abstract class LearningEntity {
  final String id;
  final String title;
  final int difficultyTier;     // 1 = Simple (Basic), 2 = Moderate (Applied), 3
= Hard (Complex)
  final int priorityScore;      // 1..100 (Computed from MoE Blueprint Weight +
Exam Frequency)
  final String priorityTier;    // "P1_CRITICAL" | "P2_HIGH_YIELD" |
"P3_SUPPLEMENTARY"
  final List<String> tags;
  final Map<String, dynamic> customAttributes; // Dynamic schema extensibility
  final DateTime updatedAt;
}

class StudyMaterial extends LearningEntity {
  final String courseId;
  final String chapterId;
  final int chapterNumber;
  final ContentStyle style;     // CONCEPT_NOTE | QUICK_SUMMARY |
CODE_WALKTHROUGH | FORMULA_SHEET | FLASHCARD
  final List<ContentBlock> blocks;             // Polymorphic: MarkdownBlock,
CodeBlock, DiagramBlock
  final List<ContentRelation> relationships;   // PREREQUISITE_OF,
RELATED_CONCEPT, TESTED_BY_QUESTION
}

class Question extends LearningEntity {
  final String courseId;
  final String chapterId;
  final int? examYearEC;        // e.g., 2015, 2016, 2017, 2018 E.C. (or null
for topic drill)
  final ExamSourceType source;  // NATIONAL_EXIT_EXAM | UNIVERSITY_MOCK |
CHAPTER_PRACTICE
  final BloomDomain domain;     // COGNITIVE | PSYCHOMOTOR | AFFECTIVE
  final String stemBody;
  final String? codeSnippet;
  final List<QuestionOption> options;          // Composed 1:N options (A, B, C,
D)
  final String generalExplanation;
  final String? linkedMaterialId;              // Direct pointer to the
StudyMaterial that explains this item
}

class QuestionOption {
  final String optionKey;       // "A" | "B" | "C" | "D"
  final String optionText;
  final bool isCorrect;
  final String rationale;       // Explains why this specific option is correct
or why it is a distractor
}
```

4. Multi-Dimensional Retrieval, Categorization & Filtering Engine

Because students study differently at different stages of exit exam preparation (Haramaya University Exit Exam Study)—starting with chapter-by-chapter concept notes, moving to simple confidence-building questions, and finishing with high-priority past-year national exam papers—the repository layer supports **combinatorial multi-faceted filtering** across all dimensions simultaneously:

Retrieval Dimension	Indexed Filter Attributes	Supported Categories & Styles	Student Use Case
1. Curriculum Hierarchy	`theme_id`, `course_id`, `chapter_id`, `title`	5 Themes → 13 Courses → Chapter No. → Topic Title	Browse all materials or questions for a specific course (e.g., *Database Systems, Chapter 3*).
2. Material Style & Format	`content_style`, `block_type`	`ConceptNote`, `QuickSummary`, `CodeWalkthrough`, `FormulaSheet`, `Flashcard`	Switch between deep textbook notes and 2-minute quick-review summary cards before an exam.
3. Content Relationship Graph	`relation_type`, `source_id`, `target_id`	`PREREQUISITE_OF`, `RELATED_CONCEPT`, `TESTED_BY_QUESTION`, `EXPLAINS_ANSWER_FOR`	Click *"Review Concept Note"* inside a missed question to jump straight to the exact chapter paragraph.
4. Exam Year & Source	`exam_year`, `source_type`	`2015 E.C.`, `2016 E.C.`, `2017 E.C.`, `2018 E.C.`, `University Mock`, `Practice Bank`	Practice official past national exit exam papers by year or compare university model exams.
5. Simplicity & Hardness	`difficulty_tier` (`1..3`), `cognitive_level`	**Tier 1 (Simple / Recall)**, **Tier 2 (Moderate / Applied)**, **Tier 3 (Hard / Analytical)**	Start with *Simple* items to build confidence, then progress to *Hard* scenario-based questions.
6. Priority & Exam Weight	`priority_tier`, `priority_score` (`1..100`)	**P1: Must-Know High-Frequency**, **P2: Core Blueprint**, **P3: Extended**	Focus last-week revision on high-priority chapters that carry the highest national blueprint share.

5. Search, Multi-Key Sorting, Navigation & Indexing Specification

To ensure that combining any filter, full-text search query, sort order, and navigation transition executes in **under 5 milliseconds** on mobile devices, the storage engine combines composite B-Tree / Document indexes (Database Indexing Strategies) with an inverted full-text search index (SQLite FTS5 Extension):

A. Full-Text Search Engine (BM25 Relevance + Prefix Matching)

- **Unified Search Virtual Index (`search_index_fts5`):** Indexes `course_title`, `chapter_title`, `material_title`, `summary_text`, `question_stem`, `option_texts`, and `tags` using the `unicode61` tokenizer with prefix indexing (`prefix='2 3 4'`).

- **Instant Scoped Search:** Students can search globally (e.g., `"OSPF routing"` or `"3NF normalization"`) or combine a text search with active category filters (e.g., search `"deadlock"` within *Operating Systems • Hard Difficulty • 2016 E.C. Exam*), ranked by `bm25()` relevance weighted by `priority_score`.

B. Multi-Criteria Sorting & Four-Mode Navigation Architecture

Mode / Capability	Query / Ordering Expression	Behavior in EthioIT ExitPrep
Sort: Priority First	`ORDER BY priority_score DESC, difficulty_tier ASC`	Surfaces the highest-yield blueprint topics first, starting from simple to hard within each priority band.
Sort: Simplicity → Hardness	`ORDER BY difficulty_tier ASC, priority_score DESC`	Progressive confidence ladder: presents foundational (Simple) material first before complex (Hard) items.
Sort: Hardness → Simplicity	`ORDER BY difficulty_tier DESC, exam_year DESC`	Challenge mode for advanced revision: surfaces the hardest recent exit exam questions first.
Sort: Exam Year & Syllabus	`ORDER BY exam_year DESC, course_code ASC, chapter_number ASC`	Orders questions chronologically from the newest national exit exam down to earlier years.
Nav 1: Breadcrumb Tree	`Theme → Course → Chapter → Section`	Hierarchical drill-down with instant chapter jump bar and completion percentage badges.
Nav 2: Graph Link Jump	`JOIN content_relations ON source_id = :id`	One-tap bidirectional navigation between a question, its underlying study note, and prerequisite chapters.
Nav 3: Smart Priority Queue	`ORDER BY (priority_score * (1 - mastery_pct)) DESC`	Dynamic playlist that automatically routes the student to high-priority chapters where their accuracy is lowest.

C. Required Composite Indexes for Sub-5ms Retrieval

```
-- 1. Multi-Faceted Question Retrieval Index (Course + Chapter + Difficulty +
Priority + Exam Year)
CREATE INDEX idx_questions_faceted
ON questions (course_id, chapter_id, difficulty_tier, priority_score DESC,
exam_year DESC);

-- 2. Exam Year & Priority Filter Index (For Past-Paper & High-Yield Mock
Generation)
CREATE INDEX idx_questions_year_priority
ON questions (exam_year DESC, priority_tier, difficulty_tier, course_id);

-- 3. Study Material Category, Style, Difficulty & Priority Index
CREATE INDEX idx_materials_lookup
ON study_materials (course_id, chapter_id, content_style, difficulty_tier,
priority_score DESC);

-- 4. Bidirectional Content Relationship Graph Index (Forward & Reverse
Traversal)
CREATE INDEX idx_relations_forward ON content_relations (source_type, source_id,
relation_type, relevance_weight DESC);
CREATE INDEX idx_relations_reverse ON content_relations (target_type, target_id,
relation_type);

-- 5. Full-Text Search Virtual Table (Titles, Notes, Questions, Options & Tags)
CREATE VIRTUAL TABLE curriculum_fts USING fts5(
  entity_type UNINDEXED, entity_id UNINDEXED, course_code, chapter_title,
  title, body_text, tags, tokenize = 'porter unicode61', prefix = '2 3 4'
);
```